

Tiny Async Python Web Server (Learning Project)

This repository contains **three intentionally minimal async web servers written in Python**.

They exist to answer one question clearly and honestly:

┆ **"What is a web server, really — and how do frameworks like Flask or FastAPI build on it?"**

No frameworks.

No magic.

No dependencies beyond Python's standard library.

Just Python, sockets, decorators, and a browser.

What this project is (and is not)

This is:

- A **learning-focused** web server implementation
- Small enough to understand end-to-end (under 100 lines per file)
- Written using only the Python standard library
- Designed for Python programmers at **all experience levels**
- A conceptual foundation for understanding real-world web development
- A gentle introduction to **async programming** and **HTTP fundamentals**

This is *not*:

- Production-ready (please don't deploy this to the internet!)
- Secure against common attacks (no CSRF, XSS, or injection protection)
- Fast or scalable (it's optimized for clarity, not performance)
- A framework replacement (use Flask, FastAPI, or Django for real projects)

This is a **teaching tool**, not a deployment tool. Think of it as "dissecting a clock to understand how time works."

Project structure

```
project/
|
├── basic_server.py      # Version 1: Bare minimum HTTP server
├── basic_server_v2.py   # Version 2: Adds routing decorators
├── basic_server_v3.py   # Version 3: Adds CSS/static file support
├── README.md
└── static/
    ├── index.html
    ├── about.html
    └── styles.css
```

Each version builds on the previous one, adding exactly **one new concept** at a time.

Requirements

- **Python 3.10+** (for type hints and `asyncio` improvements)
 - No third-party libraries required
 - Any modern web browser (Chrome, Firefox, Safari, Edge)
-

Server 1: `basic_server.py`

Purpose

`basic_server.py` is the **lowest-level useful example** of a web server.

It demonstrates the absolute fundamentals:

- How a browser connects to a server using **TCP sockets**
- What an HTTP request actually looks like (spoiler: it's just text!)
- How a URL path maps to a decision in your code
- How an HTTP response is manually constructed from scratch
- How **async servers** handle multiple clients without blocking

There are **no decorators, no routing helpers, no abstractions**.

This is the "bare metal" version—everything else is commentary.

Key concepts explained

What is `asyncio`?

`asyncio` lets Python handle multiple tasks "at the same time" without using threads. When one task is waiting (like for a browser to send data), Python switches to handle another task. This makes web servers efficient—you can serve 100 clients with a single Python process.

What is HTTP?

HTTP is just **text** sent over a network connection. When you visit a website, your browser sends something like:

```
GET /about HTTP/1.1
Host: 127.0.0.1:8000
```

And the server responds with:

```
HTTP/1.1 200 OK
Content-Type: text/html

<h1>Hello!</h1>
```

That's it. No magic. Just text following a specific format.

How to run

```
bash

python basic_server.py
```

Open a browser and navigate to:

- <http://127.0.0.1:8000/>
- <http://127.0.0.1:8000/about>

Edit `static/index.html`, refresh the browser — changes appear immediately.

Press **Ctrl+C** to stop the server gracefully.

Conceptual flow

Browser



HTTP request (plain text over TCP)



Python reads the request text



Extract path from first line



if/elif chain decides what to send



Manually build HTTP response



Send bytes back to browser



Browser renders HTML

If you understand this file, you understand the foundation of all web servers.

Try this exercise

1. Open `basic_server.py` and find the if/elif chain
2. Add a new route for `/contact`:

python

```
elif path == "/contact":  
    body = "<h1>Contact Us</h1><p>Email: hello@example.com</p>"
```

3. Restart the server and visit <http://127.0.0.1:8000/contact>
4. Notice how you had to manually add another `elif` — this doesn't scale well!

This is the problem that version 2 solves.

Server 2: `basic_server_v2.py`

Purpose

`basic_server_v2.py` introduces exactly **one new idea**: routing decorators, like the ones used in Flask and FastAPI.

Everything else stays almost the same.

This version exists to answer the question:

"How do frameworks turn URLs into Python functions?"

What's new in version 2

- A global `routes` dictionary that maps paths $\rightarrow$ handler functions
 - A `@route("/path")` decorator to register functions
 - Functions registered as HTTP endpoints
 - Clear separation between:
 - **Framework code** (the routing system)
 - **Application code** (your route definitions)
-

Understanding decorators

A decorator is just a function that wraps another function. When you write:

```
python

@route("/")
def index():
    return b"<h1>Home</h1>"
```

Python automatically does this:

```
python

def index():
    return b"<h1>Home</h1>"

index = route("/") (index) # The decorator wraps your function
```

The `@route` decorator does something simple but powerful:

```
python

routes["/"] = index # Add to the registry
```

Now when a request comes in for `/`, the server looks up `routes["/"]` and calls `index()`.

That's the entire trick. Frameworks just formalize this pattern.

Example route definition

```
python

@route("/")
def index() -> bytes:
    return load_html_file("index.html") or b"<h1>Missing index.html</h1>"
```

Breaking this down:

- `@route("/")` registers this function for the root path
- `index()` takes no arguments (the path is already known)
- It returns `bytes` (HTTP responses are always bytes, not strings)
- If the file is missing, it returns a fallback message

How to run

```
bash

python basic_server_v2.py
```

Then visit:

- <http://127.0.0.1:8000/>
- <http://127.0.0.1:8000/about>

Routes are mapped explicitly to handler functions, just like in real frameworks.

Why this version matters

This is the moment where **frameworks stop feeling magical**.

You can now see that:

- Decorators register functions **at import time** (before the server even starts)
- Routing is just **dictionary lookup** (`routes.get(path)`)
- The if/elif chain is replaced by **dynamic function calls**
- Frameworks mostly:
 - Organize code cleanly
 - Validate input
 - Protect against edge cases
 - Provide nicer APIs

They don't change the fundamentals—they just make them easier to work with.

Try this exercise

1. Add a new route to `basic_server_v2.py`:

```
python
```

```
@route("/contact")
def contact() -> bytes:
    return b"<h1>Contact</h1><p>Reach us at hello@example.com</p>"
```

2. Restart the server and visit <http://127.0.0.1:8000/contact>
 3. Notice how clean this is compared to adding another `elif`!
 4. Create `static/contact.html` and serve it instead of hardcoded HTML
-

Server 3: `basic_server_v3.py`

Purpose

`basic_server_v3.py` adds support for **CSS files** and introduces the concept of **MIME types** (telling the browser what kind of file you're sending).

This version answers:

"How do servers handle different file types like HTML, CSS, JavaScript, and images?"

What's new in version 3

- **Static file serving** for any file type (not just HTML)
 - **MIME type detection** — sending `Content-Type: text/css` for CSS files
 - **File extension routing** — automatically serving `.css` files
 - **Proper separation** between HTML routes and static assets
-

Understanding MIME types

When a browser receives a response, it needs to know what kind of file it is:

- `text/html` → render as a webpage
- `text/css` → apply as stylesheet
- `image/png` → display as an image
- `application/json` → parse as JSON data

Without the correct `Content-Type` header, browsers might display CSS as plain text or refuse to apply styles.

How CSS serving works

When your HTML contains:

```
html

<link rel="stylesheet" href="/styles.css">
```

The browser makes a **second request** to the server for `/styles.css`.

The server detects the `.css` extension and:

1. Loads `static/styles.css` from disk
2. Sets `Content-Type: text/css`
3. Sends the file bytes back to the browser
4. Browser applies the styles to the page

This is the same pattern for JavaScript files, images, fonts, etc.

How to run


```
bash
```

```
python basic_server_v3.py
```

Create `static/styles.css`:

```
css
```

```
body {  
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
  max-width: 800px;  
  margin: 0 auto;  
  padding: 20px;  
  background-color: #f5f5f5;  
}  
  
h1 {  
  color: #333;  
  border-bottom: 3px solid #007acc;  
  padding-bottom: 10px;  
}
```

Update `static/index.html` to include:

```
html
```

```
<link rel="stylesheet" href="/styles.css">
```

Visit <http://127.0.0.1:8000/> and see your styled page!

Try this exercise

1. Add more CSS rules to `styles.css` (colors, fonts, spacing)
 2. Add a second CSS file `dark-mode.css` and link it in `about.html`
 3. Try adding an image to the `static/` folder
 4. Extend the code to serve `.png` files with `Content-Type: image/png`
-

How this maps to real frameworks

This project	Flask / FastAPI	What it does
<code>asyncio.start_server</code>	ASGI / WSGI server	Accepts TCP connections
<code>handle_client()</code>	Request lifecycle	Processes each request
<code>routes</code> dict	Router object	Maps URLs to handlers
<code>@route()</code>	<code>@app.route()</code> / <code>@app.get()</code>	Registers endpoints
Handler returns bytes	Response objects	Constructs HTTP response
<code>load_html_file()</code>	Template rendering	Generates dynamic HTML
File extension checking	Static file middleware	Serves CSS, JS, images
<code>Content-Type</code> header	MIME type detection	Tells browser how to handle files

The core concepts are **exactly the same**—real frameworks just add convenience, security, and performance optimizations.

What's intentionally missing

To keep the learning surface small, this project does **not** include:

- **Query strings** (`?name=value`)
- **POST requests** and form data
- **JSON parsing** and APIs
- **Full MIME type detection** (only handles CSS explicitly)
- **Middleware** (authentication, logging, CORS)
- **Error handling frameworks** (proper 500 errors, stack traces)
- **Template engines** (Jinja2, etc.)
- **Hot reload** (auto-restart on file changes)
- **Database connections**
- **Sessions and cookies**

Each of these is a separate layer that real frameworks add. They're important for production apps but would distract from understanding the core concepts.

Suggested learning path

For absolute beginners:

1. Read `basic_server.py` **end-to-end** — don't run it yet, just read
2. Look up any unfamiliar terms (`asyncio`, `bytes`, `TCP`)
3. Run it and visit the URLs in your browser
4. Open your browser's Developer Tools (F12) and look at the Network tab
5. See the actual HTTP requests and responses — they're just text!

For intermediate learners:

1. Compare `basic_server.py` and `basic_server_v2.py` **side-by-side**
2. Focus on the `@route` decorator — how does it register functions?
3. Print the `routes` dictionary to see what's registered
4. Add a new route with a custom handler
5. Compare `@route` to Flask's `@app.route` — notice the similarities?

For advanced learners:

1. Read all three versions in sequence
 2. Add support for images (`.png`, `.jpg`) with proper MIME types
 3. Implement query string parsing (`/search?q=python`)
 4. Add POST request handling for forms
 5. Compare your code to Flask's source code on GitHub
 6. Build a tiny JSON API that returns data instead of HTML
-

Common questions

Why use `asyncio` instead of regular Python?

Regular (synchronous) Python blocks while waiting for I/O:

```
python
data = file.read() # Python stops here until done
```

Async Python can do other work while waiting:

```
python
```

```
data = await file.read() # Python can handle other requests while waiting
```

For a web server handling 100 clients, async means you can serve all of them concurrently without 100 separate threads.

Why return `bytes` instead of `str`?

HTTP is a binary protocol. Everything sent over the network must be bytes. Python strings are text (Unicode), so we explicitly convert:

```
python
```

```
response = "HTTP/1.1 200 OK\r\n".encode("utf-8") # str → bytes
```

What's the difference between `127.0.0.1` and `localhost`?

They're the same thing! `127.0.0.1` is the IP address, `localhost` is a friendly name that resolves to it. Both mean "this computer."

Can I deploy this to the internet?

Please don't! This server has no security, no error handling, and no performance optimizations. It's designed for learning, not production.

For real projects, use:

- **Flask** or **FastAPI** for Python web apps
- **nginx** or **Apache** for serving static files
- **Docker** for containerization
- **Cloud platforms** like Heroku, AWS, or Vercel

Why not just use Flask?

You should! But understanding what Flask does "under the hood" makes you a better developer. When something breaks, you'll know where to look.

Think of it like learning to drive:

- You don't need to understand the engine to drive a car
 - But mechanics need to understand engines to fix cars
 - This project teaches you to be a "web development mechanic"
-

Final note

If you understand these three files, you understand:

- **What a web server is** (a program that responds to HTTP requests)
- **How requests become responses** (read text, process it, send text back)
- **How routing works** (map URLs to functions using dictionaries)
- **Why frameworks exist** (to handle the boring/dangerous parts so you can focus on your app)
- **How static files are served** (read from disk, set correct Content-Type, send bytes)

Everything else in web development—authentication, databases, templates, APIs—is just layers built on top of these fundamentals.

Congratulations! You now know more about web servers than 90% of developers who only use frameworks.



Next steps

Ready to level up? Try building:

1. **A tiny blog** with hardcoded posts in a Python list
2. **A calculator API** that returns JSON instead of HTML
3. **A file upload handler** that saves files to disk
4. **A real-time chat app** using WebSockets (extension of HTTP)

Or dive into a real framework:

- **Flask** — minimal, flexible, great for learning
- **FastAPI** — modern, fast, built on async like this project
- **Django** — batteries-included, powerful, best for large apps

You've got the foundation. Now go build something awesome! 